Traffic Anomaly Detection with Machine Learning

Lecture by Assist. Prof. Dr. Chanankorn Jandaeng

Walailak University

Duration: 90 minutes

Objective: Understand flow-based network data, perform feature engineering, and detect anomalies using **Isolation Forest** with **Python + Pandas**.

Lecture Outline

1. Introduction: Why Traffic Anomaly Detection?
2. Network Flow Data Concepts
3. Feature Engineering for Traffic ML
4. Anomaly Detection Paradigms
5. Isolation Forest Algorithm
6. Python Demonstration — Flow Data Analysis
7. Evaluation Metrics
8. Integration with Network Monitoring
9. Summary & Lab Assignment

1. Introduction: Why Anomaly Detection?

Motivation:

- Traditional signatures (IDS/IPS) detect known attacks.
- Modern threats use **unknown behaviors** → need *behavioral / anomaly detection*.

Goal: Learn what “normal” traffic looks like and flag deviations.

Examples:

- Sudden surge of outbound connections
- Abnormal packet size distribution
- New protocols in restricted networks
- Lateral movement between hosts

2. Network Flow Data

Flow-based data:

Aggregated connection summaries between two endpoints within a time window.

Common formats:

- NetFlow / IPFIX (Cisco, standard)
- sFlow (sample-based)
- Zeek (conn.log)
- Custom PCAP → flow extraction (Wireshark / Tshark)

2. Network Flow Data

Flow fields:

Field	Description
SrcIP, DstIP	Endpoints
SrcPort, DstPort	Service identifiers
Protocol	TCP/UDP/ICMP
StartTime, Duration	Flow timing
Bytes, Packets	Volume metrics
Flags	TCP flag summary
Direction	Inbound / Outbound

Flow Data Example (Zeek or NetFlow CSV)

SrcIP	DstIP	SrcPort	DstPort	Proto	Bytes	Pkts	Duration	Fla
192.168.1.10	8.8.8.8	51342	53	UDP	80	1	0.001	-
192.168.1.20	192.168.1.1	50234	80	TCP	5400	10	3.2	S
192.168.1.30	10.0.0.5	44321	22	TCP	2000	5	1.2	S

3. Feature Engineering for Flow Analysis

Transform flow records → ML-ready numerical features

Quantitative Features

- `bytes_per_second = Bytes / Duration`
- `pkts_per_second = Packets / Duration`
- `avg_pkt_size = Bytes / Packets`
- `flow_duration`
- `unique_dst_ports`, `unique_dst_ips` per host (aggregated)

Categorical Encoding

- Protocols: TCP=1, UDP=2, ICMP=3
- One-hot encoding for protocol or service (e.g., port ranges)

3. Feature Engineering for Flow Analysis

Temporal Features

- Hour of day → diurnal traffic patterns
- Flow inter-arrival times
- Rolling statistics (mean, std, z-score)

Objective: Build features that capture **normal communication patterns**.

4. Types of Anomaly Detection

Approach	Example	Characteristics
Supervised	SVM, RF (labeled normal vs attack)	Needs labeled data
Unsupervised	Isolation Forest, One-Class SVM, Autoencoder	Learns “normal”, flags outliers
Semi-supervised	Train on normal, test on mixed	Balance between both

For real-world traffic, **unsupervised** is preferred (labels rare).

5. Isolation Forest — Concept

Idea:

Anomalies are *few* and *different* → easier to isolate.

Algorithm:

1. Randomly select a feature and split value → partition space.
2. Build many random trees (ensemble).
3. Average tree depth = anomaly score.
    - Shorter path → more anomalous.

Advantages:

- Unsupervised
- Fast on large datasets
- Handles high-dimensional numeric features

Isolation Forest — Intuition

Normal traffic: dense cluster → deep in tree.

Abnormal traffic: isolated → shallow depth.

Each tree isolates random points → combined anomaly score.

6. Python Demonstration — Flow Analysis

We'll simulate simple flow data and detect anomalies.

```
# Defensive analysis demo only
# pip install pandas scikit-learn matplotlib

import pandas as pd
from sklearn.ensemble import IsolationForest
import matplotlib.pyplot as plt

# 1. Example synthetic flow dataset
data = {
    'bytes_per_sec': [100, 120, 90, 110, 8000, 105, 95, 115, 10000, 98],
    'pkts_per_sec': [5, 6, 4, 5, 400, 6, 4, 5, 450, 5],
    'duration': [1, 1.1, 1.0, 0.9, 0.5, 1.0, 1.2, 1.0, 0.4, 1.0]
}
df = pd.DataFrame(data)
```

6. Python Demonstration — Flow Analysis

```
# 2. Train Isolation Forest
model = IsolationForest(contamination=0.2, random_state=42)
model.fit(df)
df['anomaly_score'] = model.decision_function(df)
df['is_anomaly'] = model.predict(df)
```

6. Python Demonstration — Flow Analysis

```
# 3. Display
print(df)

# 4. Visualize results
plt.scatter(df.index, df['bytes_per_sec'], c=df['is_anomaly'].map({1:'blue', -1:'red'}))
plt.title("Traffic Anomaly Detection (Red = Anomaly)")
plt.xlabel("Flow Index")
plt.ylabel("Bytes/sec")
plt.show()
```

✅ Blue = normal, Red = anomaly (high throughput or abnormal duration).

7. Evaluation Metrics for Anomaly Detection

Since labels are often unknown:

- **Precision@k**: proportion of true anomalies among top-k highest scores.
- **ROC-AUC** (if labels available).
- **Manual inspection rate:** analyst validation of top anomalies.
- **Operational metrics:** alert volume, false positive rate, analyst workload.

Interpretation:

Isolation Forest provides relative scores — threshold tuning is required.

8. Integration with Network Monitoring

Pipeline Architecture:

```
[PCAP/NetFlow] → [Flow Aggregator] → [Feature Extraction] → [Anomaly Model]
                                                ↓
                                    [Alert / Dashboard / SIEM]
```

Possible integrations:

- Zeek + ML model (Python script, Jupyter, or ELK pipeline).
- PyShark / Pandas for periodic offline analysis.
- Real-time: scikit-multiflow, Kafka streaming.

8. Integration with Network Monitoring

Visualization Tools:

- Matplotlib / Seaborn for statistical plots
- ELK (Elastic Stack) for dashboards
- Grafana / Prometheus for real-time anomaly graphs

9. Practical Lab Exercise

Objective

Analyze network flow data and identify anomalies using Isolation Forest.

Dataset

Use Zeek `conn.log` or NetFlow CSV (sanitized).

9. Practical Lab Exercise

Steps

1. Load dataset using Pandas.
2. Engineer features (`bytes_per_sec` , `pkts_per_sec` , `duration` , `protocol`).
3. Normalize numeric columns.
4. Train Isolation Forest model.
5. Visualize anomalies with scatter/box plots.
6. Interpret results — what patterns are anomalous?

Optional: Compare with One-Class SVM or Local Outlier Factor.

10. Example Feature Engineering Code (Expanded)

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest

# Load flow CSV (Zeek conn.log converted)
df = pd.read_csv("flows.csv")
```

10. Example Feature Engineering Code (Expanded)

```
# Feature engineering
df['bytes_per_sec'] = df['Bytes'] / (df['Duration'] + 1e-5)
df['pkts_per_sec'] = df['Pkts'] / (df['Duration'] + 1e-5)
df['avg_pkt_size'] = df['Bytes'] / (df['Pkts'] + 1e-5)

features = ['bytes_per_sec', 'pkts_per_sec', 'avg_pkt_size']
X = StandardScaler().fit_transform(df[features])

# Train Isolation Forest
iso = IsolationForest(n_estimators=200, contamination=0.05, random_state=42)
df['score'] = iso.decision_function(X)
df['is_anomaly'] = iso.predict(X)
```

10. Example Feature Engineering Code (Expanded)

```
# Visualization
import matplotlib.pyplot as plt
plt.scatter(df['bytes_per_sec'], df['pkts_per_sec'],
            c=df['is_anomaly'].map({1:'blue', -1:'red'}))
plt.xlabel("Bytes/sec")
plt.ylabel("Pkts/sec")
plt.title("Network Flow Anomalies (Isolation Forest)")
plt.show()
```

✅ Safe to run on **synthetic or anonymized flow logs**.

11. Use Case Discussion

Detected anomalies may represent:

- Scanning or brute-force attempts
- Data exfiltration
- Malware beaconing (low-traffic periodic flows)
- Misconfigured hosts generating excessive packets

Next Step (future lecture):

Correlate anomaly alerts with **threat intelligence** and **flow context**.

12. Best Practices

- Always normalize features (scale difference between bytes vs duration).
- Log anomaly scores → adjust threshold based on historical baseline.
- Combine **flow anomaly detection** with **signature detection** (Snort/Zeek).
- Validate anomalies manually before escalation.
- Continuously retrain models as network evolves.

13. Summary

Concept	Description
Flow Data	Connection summaries of network traffic
Feature Engineering	Convert traffic statistics into ML features
Anomaly Detection	Identify deviations from baseline behavior
Isolation Forest	Unsupervised algorithm effective for flow anomalies
Python + Pandas	Enables exploratory, repeatable analysis
Integration	Embed models into SIEM / monitoring pipelines

End of Lecture

Assist. Prof. Dr. Chanankorn Jandaeng

Walailak University

chatchanan.ja@mail.wu.ac.th

✅ Lecture Highlights

Component	Description
Duration	90 min: 25 min theory, 45 min demo, 20 min lab
Core Tool	Python + Pandas + scikit-learn
Dataset	Zeek / NetFlow CSV (synthetic)
Algorithm	Isolation Forest (unsupervised)
Focus	Flow features, outlier detection, visualization
Next Chapter Link	Leads into “Traffic Anomaly Detection with ML + Threat Correlation”